arXiv:submit/7448394 [cs.AI] 6 Apr 2026

# Targeted Remasking: Replacing Token Editing with Token-to-Mask Refinement in Discrete Diffusion Language Models*

**Lin Yao**[1,2]
[1]School of Computer Science, Shanghai Jiao Tong University, Shanghai, 200240, China
[2]Zhongguancun Academy, Beijing, 100097, China
lin.yao@sjtu.edu.cn

## Abstract

Discrete masked diffusion language models such as LLaDA generate text through iterative denoising, where mask tokens are progressively replaced with predicted tokens. LLaDA2.1 introduced a Token-to-Token (T2T) editing mechanism that accelerates generation by directly replacing committed tokens suspected of being incorrect. However, we identify fundamental limitations of T2T editing: it couples error detection with replacement, pollutes the generation context with potentially incorrect tokens, and introduces a train–inference noise mismatch where systematic model-generated errors differ from the random perturbations seen during training. We propose *Token-to-Mask* (T2M) remasking, a training-free, drop-in replacement for T2T editing that resets suspected erroneous tokens back to the mask state, allowing the diffusion process to re-predict them under cleaner context. We design and empirically validate three complementary error detection strategies—probability-based, trigger-mirrored, and temporal-difference-based—and provide a unified theoretical analysis showing that T2M remasking purifies the generation context, converts systematic inference errors back to the model's native mask noise type, and enables delayed commitment for joint multi-position optimization. Comprehensive experiments across 12 benchmarks spanning knowledge, reasoning, mathematics, coding, and instruction following show that T2M generally improves performance on tasks requiring precise token-level output, with the largest gain on mathematics (+5.92% on CMATH). Error analysis on CMATH reveals that the dominant failure mode is *last-mile token corruption*—where correct reasoning produces a corrupted final answer—and that T2M repairs 59.4% of such cases.

## 1 Introduction

Autoregressive (AR) language models [Vaswani et al., 2017] have dominated text generation, yet their sequential left-to-right decoding is inherently serial. Discrete masked diffusion language models (dLLMs) offer a compelling alternative: they generate text by iteratively denoising a fully masked sequence, enabling parallel token prediction within each denoising step [Austin et al., 2021, Sahoo et al., 2024, Lou et al., 2024]. Recent dLLMs such as LLaDA [Nie et al., 2025] and Dream [Ye et al., 2025] have achieved performance competitive with similarly-sized AR models across a broad range of benchmarks.

While dLLMs achieve significant inference speedups through parallel decoding, this parallelism comes at a cost. Within each denoising step, token predictions are conditionally independent—each position is sampled without causal conditioning on simultaneously generated neighbors—which

*Code available at https://github.com/synsis/remasked_DLM.

Preprint.

systematically amplifies token-level inconsistencies compared to AR models [Kang et al., 2025]. This heightened error susceptibility, compounded by exposure bias in which the model conditions on its own imperfect predictions, makes error correction a critical bottleneck for generation quality [Bie et al., 2026].

To address this, LLaDA2.1 [Bie et al., 2026] introduced a key inference-time innovation: *Token-to-Token* (T2T) editing. Beyond the standard Mask-to-Token (M2T) step that fills in mask positions, T2T directly replaces already-committed (non-mask) tokens when the model predicts a different token at that position with high confidence. This mechanism targets the error correction bottleneck by enabling the model to retroactively "change its mind" about previously generated tokens. With these innovations, LLaDA2.1 has demonstrated performance competitive with leading AR models at comparable scale across diverse benchmarks [Bie et al., 2026], establishing dLLMs as a practical alternative for text generation. Improving the inference quality of such a system—without additional training cost—is therefore of direct practical significance.

However, we identify three fundamental limitations of T2T editing:

1. **Detection–replacement coupling.** T2T couples error detection and replacement into a single trigger: it fires only when the model assigns probability exceeding $\tau_{\text{t2t}}$ to a token *different* from the current one. In other words, T2T cannot express "this token is wrong" without simultaneously providing "replace it with this." When the conditional distribution is multimodal (multiple plausible tokens sharing probability mass), no single candidate may exceed the threshold, leaving clearly erroneous tokens uncorrected even though the model recognizes them as wrong—a failure mode we term *correction inertia* (Figure 1a).
2. **Context pollution** (fundamental limitation of T2T). T2T's core design—replacing one token with another—cannot guarantee context improvement. Even when T2T is triggered, its replacement is conditioned on context that may itself contain errors at other positions; under such polluted context, even high-confidence replacements may be incorrect. The replaced token, appearing as a legitimate signal, provides misleading semantic cues that bias predictions at all other positions and can create self-reinforcing error cycles (Figures 1b and 2).
3. **Train–inference noise mismatch** (limitation of LLaDA2.1's training procedure). LLaDA2.1's T2T training stream uses *random* token perturbations (uniformly sampled wrong tokens $\rightarrow$ ground truth), but at inference time, erroneous tokens are *systematic, model-generated* mistakes that are semantically plausible and correlated with surrounding tokens. This distributional gap means the model encounters a third type of context noise at inference that it has never seen during training—neither the clean `[M]` noise of the M2T stream nor the random perturbations of the T2T stream, but systematic, semantically coherent errors from the model itself (see Figure 2 for empirical evidence of how such errors mislead predictions).

We propose *Token-to-Mask* (T2M) remasking as a principled alternative. Instead of replacing suspected tokens, T2M resets them to the `[M]` state, allowing the diffusion process to re-predict these positions from scratch under a cleaner context. This simple change simultaneously addresses all three limitations: it decouples error detection from replacement, purifies the context by removing erroneous tokens, and converts systematic inference errors back to the model's native `[M]` noise type. Moreover, by reverting to `[M]` rather than immediately committing a replacement, T2M naturally implements *delayed commitment*—deferring the final token decision to a later step where the surrounding context has further converged, enabling joint re-optimization of multiple suspect positions (Figure 1c). Figure 1 illustrates the contrast: T2T leaves an error in place when no confident replacement exists (*correction inertia*), while T2M resets it to `[M]` and lets a subsequent M2T step recover the correct token under purified context.

Our approach is *training-free*—it requires no additional training or architectural modification and works as a drop-in replacement for T2T within LLaDA2.1's existing generation pipeline. We design and empirically validate three complementary error detection strategies (LOWPROB, T2T-REMASK, and LOGITDIFF), each targeting a different signal of token unreliability, with thresholds tuned on a small-scale pilot set. We provide a unified theoretical analysis from four perspectives: decoupled detection, context purification, train–inference noise mismatch, and delayed commitment.

Our contributions are:

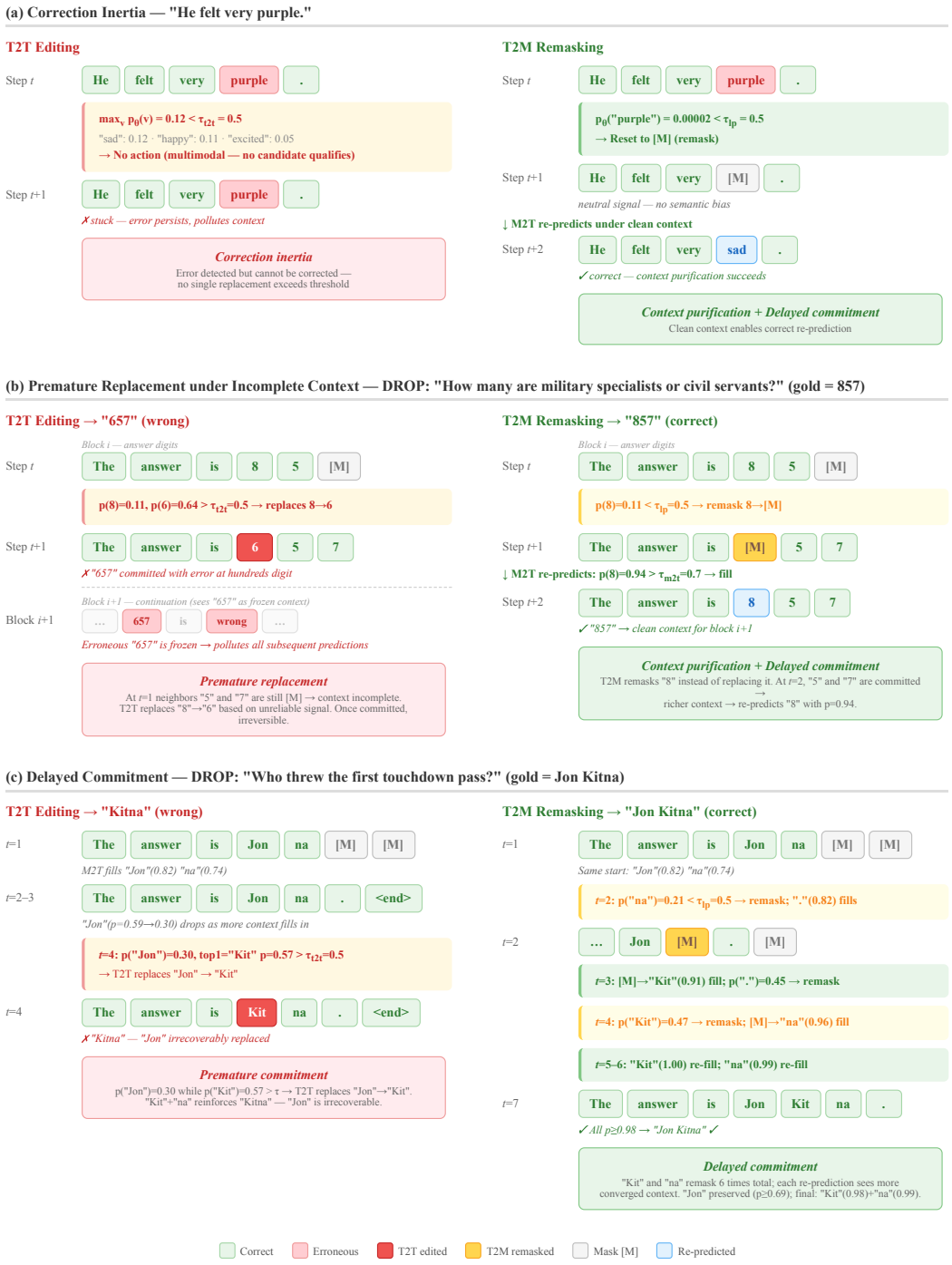


Figure 1: **T2T editing vs. T2M remasking.** All probabilities are from LLaDA2.1-mini. **(a) Correction inertia**: "He felt very purple."—T2T detects the error ($p$(purple)=0.00002) but cannot correct it because the distribution is multimodal ("sad": 0.12, "happy": 0.11); T2M remasks it and re-predicts correctly. **(b) Premature replacement** (DROP, gold=857): T2T replaces the initially correct "8" ($p$=0.11) with "6" ($p$=0.64>$\tau_{\text{t2t}}$) under incomplete context; T2M remasks "8", waits for context convergence, then re-predicts "8" with $p$=0.94. **(c) Delayed commitment** (DROP, gold=Jon Kitna): T2T greedily replaces "Jon"→"Kit" ($p$=0.57), losing the first name; T2M iteratively remasks uncertain tokens (6 times), preserving "Jon" while "Kit" and "na" converge to $p\geq$0.98.

- We identify fundamental limitations of T2T editing in LLaDA2.1 and propose T2M remasking as a principled, training-free replacement.
- We design and empirically validate three error detection strategies on a small-scale pilot set, and analyze their complementary strengths.
- We provide a unified theoretical framework explaining why mask tokens are strictly preferable to erroneous tokens as context, from the perspectives of train–inference noise mismatch, context signal hierarchy, and delayed commitment.
- We discover that *last-mile token corruption*—where the model's reasoning is correct but the final answer is corrupted during denoising—is the dominant error mode (79.9% of errors on CMATH), and show that T2M specifically repairs this failure mode.
- We conduct comprehensive experiments across 12 benchmarks, showing general improvements on tasks requiring precise token-level output, with the largest gain on mathematics.

## 2 Background

### 2.1 Masked Diffusion Language Models

A masked diffusion language model defines a forward noising process that progressively masks tokens and a reverse denoising process that recovers them [Austin et al., 2021, Ou et al., 2025, Sahoo et al., 2024]. Given a clean sequence $\mathbf{x} = (x_1, \ldots, x_L)$, the forward process at time $t \in [0, 1]$ independently replaces each token with the mask token `[M]` with probability $\alpha_t$ (where $\alpha_0 = 0$ and $\alpha_1 = 1$), yielding the noisy sequence $\mathbf{z}_t$.

The model $p_\theta(\mathbf{x} \mid \mathbf{z}_t)$ is trained to predict all masked positions simultaneously via a cross-entropy loss:

$$\mathcal{L}(\theta) = -\mathbb{E}_{\mathbf{x},t}\left[\frac{1}{\sum_i \mathbb{1}[z_t^i = \texttt{[M]}]} \sum_{i=1}^{L} \mathbb{1}[z_t^i = \texttt{[M]}] \log p_\theta(x_i \mid \mathbf{z}_t)\right]. \tag{1}$$

At inference, generation proceeds by starting from a fully masked sequence and iteratively: (1) predicting all masked positions, (2) retaining predictions whose confidence exceeds a threshold, subject to a budget cap determined by the unmasking schedule—when more predictions qualify than the budget allows, only the top-$k$ most confident are committed and the rest are remasked [Sahoo et al., 2024].

### 2.2 Block Diffusion

LLaDA2.1 [Bie et al., 2026] adopts a *semi-autoregressive* block diffusion architecture. The response is divided into blocks of $B$ tokens (typically $B = 32$), generated *sequentially* from left to right; within each block, tokens are generated *in parallel* via iterative denoising.

**Prompt and response handling.** Given a prompt–response pair, the prompt tokens are placed into the sequence *unmasked*—only the response positions are initialized as `[M]` tokens. In blocks that partially overlap with the prompt, the prompt positions are marked as non-editable and remain frozen throughout generation.

**Block-causal attention.** To maintain left-to-right coherence while preserving parallel generation within each block, LLaDA2.1 employs a *block-causal* attention mask. *Within* a block, all positions attend to each other bidirectionally, enabling the model to capture intra-block dependencies. *Across* blocks, attention is strictly causal: block $j$ can attend to blocks $0, 1, \ldots, j$ but not to any future block $j' > j$. This block-level lower-triangular mask is constructed as $\mathbf{M}_{\text{attn}} = \text{tril}(\mathbf{1}_{N_b \times N_b})$ at the block level and expanded to token-level resolution. Previously generated blocks are frozen and their key–value states cached for efficient reuse in subsequent blocks.

**Iterative M2T denoising.** Within each block, the model iteratively fills `[M]` positions via Mask-to-Token (M2T) steps. At each iteration, the model predicts token distributions $p_\theta(x_i \mid \mathbf{z})$ for all masked positions. Positions whose confidence exceeds a threshold $\tau_{\text{m2t}}$ are committed; when fewer positions qualify than the per-step budget $n_{\text{transfer}}$, the top-$n_{\text{transfer}}$ most confident predictions are committed instead. The inner loop repeats until all masks in the block are filled.

### 2.3 Token-to-Token (T2T) Editing

Beyond M2T, LLaDA2.1 introduces a *Token-to-Token* (T2T) editing phase at each denoising step. For positions already containing committed (non-mask) tokens, the model checks whether a different token is predicted with confidence exceeding $\tau_{\text{t2t}}$. If so, the committed token is *directly replaced*:

$$x_i \leftarrow x_i^* \quad \text{if} \quad p_\theta(x_i^* \mid \mathbf{z}) > \tau_{\text{t2t}} \ \wedge \ x_i^* \neq x_i^{\text{old}}, \tag{2}$$

where $x_i^* = \arg\max_v p_\theta(v \mid \mathbf{z})$ is the model's top prediction and $x_i^{\text{old}}$ is the currently committed token. Prompt positions within a block are excluded from T2T editing.

The inner loop continues until all masks are filled and no further T2T edits are triggered. Then generation proceeds to the next block, with the current block's tokens frozen as context.

## 3 Method: Token-to-Mask Remasking

### 3.1 Overview

We replace T2T editing with *Token-to-Mask* (T2M) remasking. When the error detection mechanism identifies a committed token as likely incorrect, instead of replacing it with another token, we reset it to `[M]`:

$$x_i \leftarrow \texttt{[M]} \quad \text{if} \quad \textsc{ShouldRemask}(i, \mathbf{z}, \theta), \tag{3}$$

and the subsequent M2T step re-predicts this position with updated context. This is a minimal, surgical change to LLaDA2.1's generation loop: only the token editing phase is replaced (see Algorithm 1); all other components (model architecture, M2T logic, block scheduling, KV caching) remain identical.

### 3.2 Error Detection Strategies

We design and empirically validate three complementary strategies for SHOULDREMASK, each capturing a different signal of token unreliability.

**Strategy 1: LOWPROB — Current probability is low.** The model re-evaluates all committed tokens under the current context. If the probability assigned to the currently committed token is low, it is remasked:

$$\textsc{Remask}_{\text{LP}}(i) = \mathbb{1}\left[p_\theta(x_i^{\text{old}} \mid \mathbf{z}_{-i}) < \tau_{\text{lp}}\right]. \tag{4}$$

This strategy directly tests "does the model still think this token belongs here?" without requiring a specific replacement candidate.

**Strategy 2: T2T-REMASK — Same trigger, different action.** This strategy uses the identical triggering condition as T2T (the model confidently predicts a *different* token), but applies remasking instead of replacement:

$$\textsc{Remask}_{\text{T2T}}(i) = \mathbb{1}\left[p_\theta(x_i^* \mid \mathbf{z}) > \tau_{\text{tr}} \ \wedge \ x_i^* \neq x_i^{\text{old}}\right]. \tag{5}$$

By preserving the detection logic but changing the action, this strategy isolates the effect of mask-based correction versus token-based replacement.

**Strategy 3: LOGITDIFF — Confidence drop.** This strategy compares the model's outputs from two consecutive denoising iterations within the same block. Let $p_\theta^{(t)}(\cdot \mid \mathbf{z}^{(t)})$ denote the model's prediction at iteration $t$, where the context $\mathbf{z}^{(t)}$ reflects the block state *after* the M2T/editing updates of iteration $t-1$. For a committed token $x_i^{\text{old}}$ at position $i$, we remask when the model's confidence in that token *drops by more than* $\tau_{\text{ld}}$ between iterations:

$$\textsc{Remask}_{\text{LD}}(i) = \mathbb{1}\left[p_\theta^{(t-1)}(x_i^{\text{old}} \mid \mathbf{z}^{(t-1)}) - p_\theta^{(t)}(x_i^{\text{old}} \mid \mathbf{z}^{(t)}) > \tau_{\text{ld}}\right]. \tag{6}$$

A confidence drop indicates that, as surrounding positions get filled or corrected, the evolving context no longer supports this token—a signal that it is inconsistent with its converging neighborhood. An increase in confidence, conversely, means the token is being validated by the evolving context and should be kept. Unlike LOWPROB which uses a single-step snapshot, LOGITDIFF captures *dynamic* context evolution across iterations. On the first iteration of each block, no previous logits exist, so no remasking is triggered.

**Algorithm 1** Token-to-Mask (T2M) Remasking (replacing T2T editing in LLaDA2.1)

**Require:** Current block tokens $\mathbf{z}$, model $p_\theta$, mask id `[M]`, strategy $\mathcal{S}$, threshold $\tau$, budget $C_{\max}$, ratio cap $\rho_{\max}$

**// Standard M2T step (unchanged from LLaDA2.1)**
**for** each mask position $i$ where $z_i =$ `[M]` **do**
  **if** $\max_v p_\theta(v \mid \mathbf{z}) > \tau_{\text{m2t}}$ **then**
    $z_i \leftarrow \arg\max_v p_\theta(v \mid \mathbf{z})$
  **end if**
**end for**

**// T2M remasking step (replaces T2T editing)**
$\mathcal{E} \leftarrow \{i : z_i \neq$ `[M]` and $i \notin$ prompt$\}$ ▷ editable positions
$\mathcal{R} \leftarrow \{i \in \mathcal{E} : \textsc{ShouldRemask}_{\mathcal{S}}(i, \mathbf{z}, \tau)$ and $c_i < C_{\max}\}$
**if** $|\mathcal{R}| > \rho_{\max} \cdot |\mathcal{E}|$ **then**
  $\mathcal{R} \leftarrow$ top-$k$ by lowest confidence score, $k = \lfloor \rho_{\max} \cdot |\mathcal{E}| \rfloor$
**end if**
**for** each $i \in \mathcal{R}$ **do**
  $z_i \leftarrow$ `[M]`; $c_i \leftarrow c_i + 1$
**end for**
**return** $\mathcal{R} = \emptyset$ ▷ converged if no remasking occurred

### 3.3 Safety Mechanisms

Unrestricted remasking could lead to oscillation (repeatedly remasking and re-predicting the same positions). We employ two safeguards:

**Per-position remask budget.** Each position $i$ maintains a counter $c_i$, incremented each time it is remasked. Remasking is blocked when $c_i \geq C_{\max}$ (default $C_{\max} = 1$), preventing infinite cycles.

**Remask ratio cap.** At each step, at most a fraction $\rho_{\max}$ (default $\rho_{\max} = 0.25$) of editable positions may be remasked. When more positions are flagged, we prioritize those with the lowest confidence scores (strategy-dependent), ensuring the most suspect tokens are addressed first.

## 4 Theoretical Analysis

We provide a unified theoretical framework for understanding why T2M remasking is preferable to T2T editing. Our analysis proceeds along four complementary axes.

### 4.1 Decoupling Error Detection from Correction

T2T editing couples two logically independent decisions into a single condition: (1) *Is the current token wrong?* and (2) *Is there a confident replacement?* This coupling creates *correction inertia*: despite clear evidence that a token is erroneous, the system cannot initiate correction because no single replacement meets the confidence threshold. We formalize this via the *stuck set*:

**Definition 1** (Stuck set)**.** *The stuck set $\mathcal{S}_{\text{stuck}}$ contains positions where the current token is likely wrong but no single replacement exceeds the confidence threshold:*

$$\mathcal{S}_{\text{stuck}} = \left\{ i : p_\theta(x_i^{\text{old}} \mid \mathbf{z}_{-i}) < \epsilon, \textit{ but } \max_v p_\theta(v \mid \mathbf{z}_{-i}) < \tau_{t2t} \right\}. \tag{7}$$

This situation arises naturally when the conditional distribution $p_\theta(\cdot \mid \mathbf{z}_{-i})$ is multimodal. For instance, consider completing "He felt very _____." with LLaDA2.1-mini: the tokens "sad," "happy," and "excited" receive probabilities $0.12$, $0.11$, and $0.05$ respectively (Figure 1), while the incorrect committed token "purple" has probability $\sim$0.00002. T2T with $\tau_{\text{t2t}} = 0.5$ takes no action since $\max_v p_\theta(v) = 0.12 < 0.5$, despite the model clearly recognizing the error.

T2T is completely ineffective on $\mathcal{S}_{\text{stuck}}$: its trigger requires $\max_v p_\theta(v \mid \mathbf{z}_{-i}) > \tau_{\text{t2t}}$, which fails by definition of $\mathcal{S}_{\text{stuck}}$. In contrast, LowProb only checks whether $p_\theta(x_i^{\text{old}} \mid \mathbf{z}_{-i}) < \tau_{\text{LP}}$—it does not

require high confidence in any specific replacement. Since $p_\theta(x_i^{\text{old}} \mid \mathbf{z}_{-i}) < \epsilon \ll \tau_{\text{LP}}$ for positions in $\mathcal{S}_{\text{stuck}}$, LowProb necessarily triggers remasking, handing these positions to the subsequent M2T step for re-prediction under potentially more converged context. Note that remasking is a *necessary* step for initiating correction but does not guarantee a correct re-prediction—the key point is that T2T cannot even take this first step.

### 4.2 Error Context Purification

The most critical advantage of T2M concerns the quality of the *context* under which re-prediction occurs. In iterative denoising, the model's prediction at position $i$ is conditioned on all other positions:

$$p_\theta(x_i \mid \mathbf{z}_{-i}) = p_\theta\left(x_i \mid \underbrace{\mathbf{z}_{\text{correct}}}_{\text{correct tokens}}, \underbrace{\mathbf{z}_{\text{error}}}_{\text{wrong tokens}}, \underbrace{\texttt{[M]}, \ldots, \texttt{[M]}}_{\text{unknown}}\right). \tag{8}$$

**Erroneous tokens in context.** In parallel generation, errors are inevitable: M2T may fill a mask position with an incorrect token, and T2T may replace a token with another that is also wrong. These erroneous tokens—regardless of their source—appear as legitimate tokens in the context, providing misleading semantic signals that bias predictions at all other positions. The key question is: when an error is detected, what should be done with the position?

**T2T: Replace with another token.** T2T substitutes the suspected error with the model's current top prediction. However, even when this replacement has high confidence, the model's prediction is conditioned on context that may itself contain errors at other positions—under a polluted context, high confidence does not imply high correctness. Consequently, the replacement may itself be incorrect, and the erroneous token remains in the context as a seemingly legitimate signal, biasing not only subsequent predictions at *other* positions but also the re-evaluation of this *same* position in later iterations, creating a self-reinforcing error cycle.

**T2M: Reset to neutral mask.** T2M replaces suspected errors with `[M]`, which the model recognizes as "unknown." Mask tokens carry no semantic bias—they are entirely neutral, neither misleading predictions at other positions nor interfering with re-prediction at the reset position itself. By progressively replacing detected errors with `[M]`, the proportion of correct tokens and neutral masks in the context increases while misleading signals are correspondingly reduced.

**Remark 1** (Context signal hierarchy)**.** *For predicting position $i$, each context position $j \neq i$ provides one of three types of signals:*

1. ***Aligned signal*** *($z_j = x_j^*$, correct token): biases $p_\theta(x_i \mid \mathbf{z})$ toward the correct $x_i^*$, as the model leverages correct semantic dependencies.*
2. ***Null signal*** *($z_j =$ `[M]`): does not bias $p_\theta(x_i \mid \mathbf{z})$ in any semantic direction. The model, having been trained extensively on masked contexts, produces calibrated predictions that effectively marginalize over the unknown position.*
3. ***Adversarial signal*** *($z_j = e_j \neq x_j^*$, erroneous token): biases $p_\theta(x_i \mid \mathbf{z})$* away *from $x_i^*$. The model cannot distinguish $e_j$ from a correct token, so it trusts the wrong semantic signal and adjusts its prediction accordingly.*

Under this hierarchy, T2M replaces type (3) signals with type (2), eliminating sources of adversarial bias at zero cost (null signals are strictly non-harmful). T2T replaces type (3) with a new token that may itself be type (3), providing no guarantee of net improvement. Figure 2 provides empirical verification: we construct "I went to _____ and visited the `[M]` Tower." with four context variants and measure the model's prediction at the `[M]` position. With the correct token "France," the model predicts "Eiffel" ($p$=0.97). With `[M]`, the prediction is preserved ("Eiffel," $p$=0.82) but less confident. With the semantically related error "Japan," the model is *actively misled* to predict "Tokyo" ($p$=0.91)—a high-confidence wrong answer. This demonstrates that adversarial signals are strictly worse than null signals, and that T2M's conversion of adversarial→null is always safe.

Figure 2: **Empirical verification of context signal hierarchy** (Remark 1) on LLaDA2.1-mini. We vary a single context position [X] in "I went to [X] and visited the `[M]` Tower" and measure predictions at the `[M]` position. Aligned context (France) yields a confident correct prediction ($p$(Eiffel)=0.97). Null context (`[M]`) preserves correctness at reduced confidence (0.82). Adversarial context (Japan) *actively misleads* the model to a wrong answer ($p$(Tokyo)=0.91). Unrelated noise (banana) degrades confidence without directing to a specific wrong answer ($p$(Eiffel)=0.33). This confirms: aligned $>$ null $>$ adversarial, justifying T2M's adversarial→null conversion.

**Error propagation cycle.** When multiple adversarial signals coexist, they interact: an error at position $j$ biases the prediction at position $i$, which in turn reinforces the error at $j$ in subsequent iterations. This mutual reinforcement can converge to a *stable but incorrect fixed point*. T2M breaks this cycle by simultaneously converting multiple adversarial signals to null signals, severing the inter-position error dependencies.

### 4.3 Train–Inference Noise Mismatch

LLaDA2.1 employs a dual-stream training objective [Bie et al., 2026]: the M2T stream trains the model to predict correct tokens from masked context; the T2T stream trains the model to recover ground-truth tokens from randomly perturbed positions (random wrong token → ground truth). The model has thus encountered both `[M]` noise and random-token noise during training. However, there is a critical distributional gap between training noise and inference errors.

**Random perturbations $\neq$ systematic errors.** During T2T training, the perturbed tokens are sampled *uniformly at random* from the vocabulary—they are uncorrelated with the context and semantically incoherent. At inference time, however, erroneous tokens are *systematic, model-generated* mistakes that are often semantically plausible, locally coherent, and correlated with surrounding tokens. This gap affects *both* M2T and T2T operations: regardless of the operation type, the non-mask context during inference may contain model-generated errors whose distribution differs fundamentally from anything seen during training.

**Mask as a strictly safer context signal.** Given this mismatch, what is the best action when an error is detected? Mask tokens carry strictly less misleading information than erroneous tokens: a mask

communicates "unknown," whereas an erroneous token actively provides a false semantic signal. By converting suspected errors to [M]:

$$\underbrace{\{\mathbf{z}_{\text{correct}}, \mathbf{z}_{\text{error}}\}}_{\text{context with errors}} \xrightarrow{\text{T2M}} \underbrace{\{\mathbf{z}_{\text{correct}}, \texttt{[M]}\}}_{\text{cleaner context}}, \tag{9}$$

T2M replaces actively misleading signals with neutral "unknown" markers, converting the context back to a form the model handles most reliably. Figure 2 illustrates this mismatch concretely: random perturbations during T2T training resemble "banana" replacing "France"—semantically incoherent noise that merely degrades confidence ($p(\text{Eiffel})=0.33$) without directional bias. In contrast, actual inference-time errors resemble "Japan" replacing "France"—semantically related, locally coherent errors that *actively mislead* the model toward high-confidence wrong predictions ($p(\text{Tokyo})=0.91$). This distributional gap is the essence of the train–inference mismatch. [M] avoids both failure modes entirely ($p(\text{Eiffel})=0.82$).

**Analogy with continuous diffusion.** In continuous diffusion models [Ho et al., 2020, Song et al., 2021], the denoising score function $\nabla_{\mathbf{x}} \log p(\mathbf{x})$ is trained with Gaussian noise. If the noise distribution at test time deviates from the training distribution (e.g., structured adversarial perturbations instead of random Gaussian noise), score estimation degrades. Analogously, in masked diffusion, the model is trained with two noise types: [M] tokens (M2T) and random token perturbations (T2T). Inference-time errors—systematic, correlated, semantically plausible—constitute a third, unseen noise type (see Figure 2). T2M addresses this by converting the unseen noise back to the familiar [M] type.

### 4.4 Delayed Commitment and Joint Optimization

T2T is a *greedy* strategy: once it identifies a high-confidence alternative, it immediately commits. This greedy commitment has two problems:

1. **Premature locking.** The replacement token, once committed, becomes context for all other positions. If the replacement is itself suboptimal (e.g., the argmax under multimodal distributions), this error propagates irreversibly.
2. **Per-position independence.** T2T optimizes each position independently: $\hat{x}_i = \arg\max p_\theta(x_i \mid \mathbf{z}_{-i})$. This ignores inter-position dependencies.

T2M implements *delayed commitment*: by resetting suspect positions to [M], it defers the final decision to a later step where more context has converged. When multiple positions are simultaneously remasked, the subsequent M2T step effectively solves a *joint* optimization:

$$\hat{\mathbf{x}}_{\mathcal{R}} = \arg\max_{\mathbf{x}_{\mathcal{R}}} \prod_{i \in \mathcal{R}} p_\theta(x_i \mid \mathbf{z}_{\text{correct}}, \texttt{[M]}^{|\mathcal{R}|}), \tag{10}$$

where $\mathcal{R}$ is the set of remasked positions. While the prediction at each position is still independent conditional on the context, the context itself is *jointly cleaner* (all errors in $\mathcal{R}$ are simultaneously removed), yielding better individual predictions that are more globally consistent.

This is analogous to *simulated annealing*: T2M locally increases uncertainty (remasking) to escape local optima and explore more globally optimal solutions.

**Illustrative examples.** Figure 1(b) demonstrates premature locking on a DROP example (gold answer = 857). At an early denoising step, when only "8" ($p$=0.11) has been committed while "5" and "7" are still [M], T2T greedily replaces "8" with "6" ($p=0.64 > \tau_{\text{t2t}}$) based on incomplete context, producing the wrong answer 657. T2M instead remasks "8", waits for "5" and "7" to be committed, and then re-predicts "8" with $p$=0.94 under fully converged context.

Figure 1(c) illustrates joint multi-position optimization on a DROP example (gold = "Jon Kitna"). T2T greedily replaces "Jon"→"Kit" ($p$=0.57), destroying the first name. T2M iteratively remasks uncertain tokens (6 rounds), allowing "Jon" to remain stable while "Kit" and "na" gradually converge to $p \geq 0.98$—the system finds a globally consistent solution by deferring commitment across multiple interdependent positions.

## 5 Related Work

**Discrete masked diffusion language models.** Masked diffusion models for text generation have advanced rapidly. D3PM [Austin et al., 2021] introduced discrete diffusion with absorbing states. MDLM [Sahoo et al., 2024] and SEDD [Lou et al., 2024] developed efficient training objectives. LLaDA [Nie et al., 2025] scaled to 8B parameters with competitive performance, and LLaDA2.1 [Bie et al., 2026] introduced T2T editing for faster inference. Dream [Ye et al., 2025] further demonstrated the viability of diffusion LMs at scale.

**Remasking in masked diffusion.** ReMDM [Wang et al., 2025] is the pioneering work on remasking for masked diffusion. It modifies the reverse posterior to introduce a *probabilistic* remasking parameter $\sigma_t$, allowing any unmasked token to be randomly remasked at each step. ReMDM operates at the *sampling process level*, modifying the entire reverse diffusion trajectory. In contrast, our approach operates specifically at the *editing step level* within LLaDA2.1, performing *targeted* remasking based on explicit error detection rather than random remasking.

Using the signal hierarchy of Remark 1, we can formalize why targeted remasking is strictly preferable. Consider $N$ committed (non-mask) positions, of which $N_c$ are correct (aligned signal, value $s_+ > 0$) and $N_e$ are erroneous (adversarial signal, cost $s_- < 0$). Define context quality as $Q = N_c \cdot s_+ + N_e \cdot s_-$. Under **random remasking** with per-token probability $\sigma$, each position is independently remasked regardless of correctness:

$$Q_{\text{random}}(\sigma) = (1-\sigma)(N_c \cdot s_+ + N_e \cdot s_-). \tag{11}$$

This creates an inherent dilemma: increasing $\sigma$ removes adversarial signals but *equally* destroys aligned signals. Under **targeted remasking** (assuming perfect detection), only the $N_e$ erroneous positions are remasked:

$$Q_{\text{targeted}} = N_c \cdot s_+. \tag{12}$$

The advantage of targeted over random remasking is:

$$Q_{\text{targeted}} - Q_{\text{random}}(\sigma) = \underbrace{\sigma \cdot N_c \cdot s_+}_{\text{aligned signals preserved}} + \underbrace{(1-\sigma)(-N_e \cdot s_-)}_{\text{adversarial signals removed}} > 0 \tag{13}$$

for all $\sigma \in (0, 1]$. Both terms are strictly positive: targeted remasking simultaneously preserves more correct context and removes more errors. Even with imperfect detection, targeted remasking outperforms random remasking as long as the detector is better than chance, i.e., its precision-recall tradeoff exceeds the base rate $N_e/N$.

CORE [Zhai et al., 2026] proposes context-robust remasking that identifies "context-brittle" tokens via sensitivity to masked perturbations, arguing that static probability-based heuristics are myopic. However, we note a potential limitation of the sensitivity-based approach. CORE's sensitivity metric measures conditional mutual information $I(X_i; Z_S \mid Z_{-i,-S})$, which may be influenced by the *current mask pattern* of the denoising state rather than token correctness alone. For instance, a correct token may exhibit low sensitivity when its semantically related neighbors are still `[M]` (the relevant context is already absent), yet the same token's sensitivity may spike once those neighbors are filled—regardless of whether the fill is correct. Conversely, an erroneous token may appear "stable" simply because its correlated neighbors happen to be unpredicted. This suggests that CORE's sensitivity may partly reflect the *transient state of the denoising process* (which positions happen to be filled) rather than the intrinsic correctness of the token.

In contrast, the static probability $p_\theta(x_i \mid \mathbf{z})$ is the model's *posterior belief* about $x_i$ given the *full* current context—arguably the most direct available signal for token correctness. By the data processing inequality, evaluating under artificially degraded context (CORE's masking perturbations) cannot yield strictly more information about $X_i$'s correctness than the full-context prediction.

Practically, CORE requires $O(k)$ additional forward passes per perturbation test, while static probability is a free byproduct of the existing forward pass. Our LogitDiff strategy captures a related but more principled form of sensitivity: it tracks how the model's opinion changes along the *actual* denoising trajectory rather than under artificial perturbations, also at zero extra forward-pass cost.

**Training-based correction.** Several methods improve remasking quality through additional training. RemeDi [Huang et al., 2025] trains a dual-stream architecture with supervised fine-tuning and

reinforcement learning to learn per-token confidence scores for remasking decisions. ProSeCo [Schiff et al., 2026] adds a corrector cross-entropy loss to train the model to recover from its own mistakes. PRISM [Kim et al., 2025] attaches a lightweight adapter fine-tuned with a self-correction loss. MDPO [He et al., 2025] frames denoising as sequential decision-making and applies policy optimization, also proposing Running Confidence Remasking (RCR) as a training-free baseline. These methods are complementary to our approach: they improve the model's ability to detect or correct errors through training, while we improve the *correction mechanism* itself (T2M vs. T2T) without any training.

**Positioning.** Our approach offers principled advantages over existing remasking methods along three axes. *Compared with ReMDM*, we replace blind random remasking with targeted error-driven remasking, which we formally show to be strictly superior in context quality (Eq. 11–13). *Compared with CORE*, we use direct posterior probability and natural denoising-trajectory dynamics (LOGITDIFF), both grounded in the model's full-context prediction, rather than perturbation-based sensitivity metrics that may be influenced by the transient mask pattern. *Compared with training-based methods* (RemeDi, ProSeCo, PRISM, MDPO), we achieve improvements at zero training cost by addressing the correction mechanism itself (T2M vs. T2T) rather than learning better error detectors. Our theoretical framework—the signal hierarchy (Remark 1), the targeted-vs-random analysis, and the delayed commitment formulation—provides a unified foundation that is complementary to and composable with all of the above.

# 6 Experiments

## 6.1 Experimental Setup

**Model.** We use LLaDA2.1-mini (16B parameters) [Bie et al., 2026], a Mixture-of-Experts masked diffusion language model. All experiments use the same pretrained checkpoint with no additional training.

**Inference hyperparameters.** We adopt LLaDA2.1's recommended Quality Mode (Q Mode) settings: M2T confidence threshold $\tau_{\text{m2t}} = 0.7$, T2T editing threshold $\tau_{\text{t2t}} = 0.5$, block length $B = 32$, temperature $= 0.0$ (greedy), and `eos_early_stop` = True. Generation length is set per-benchmark (see Appendix A).

**Remask hyperparameters.** For the main results, we use T2M with the LOWPROB strategy ($\tau_{\text{LP}} = 0.3$, $C_{\max} = 1$, $\rho_{\max} = 0.25$), selected as the most cost-efficient configuration from the ablation study (Figure 3). The full hyperparameter sweep across all three strategies is presented in the ablation.

**Baselines.** Our primary comparison is between the original T2T editing ("Original") and T2M remasking with the configuration described above. All methods use the same model and inference parameters, isolating the effect of the editing mechanism.

**Evaluation note.** We use the official LLaDA2.1 model weights, the HuggingFace-provided inference code, and official benchmark datasets. As LLaDA2.1 does not release its evaluation code, our reproduced baseline absolute scores may differ from the original paper due to prompt template or answer extraction differences; all T2T vs. T2M comparisons use *identical* evaluation pipelines.

**Benchmarks.** We evaluate on 12 benchmarks spanning five categories: **Knowledge** (GPQA-Diamond [Rein et al., 2024], TriviaQA [Joshi et al., 2017], MMLU-Pro [Wang et al., 2024]), **Reasoning** (HellaSwag [Zellers et al., 2019], PIQA [Bisk et al., 2020], DROP [Dua et al., 2019], BBH [Suzgun et al., 2023]), **Mathematics** (CMATH [Wei et al., 2023], AIME 2025 [Mathematical Association of America, 2025], GSM Plus [Li et al., 2024]), **Coding** (HumanEval+ [Chen et al., 2021, Liu et al., 2023]), and **Instruction Following** (IFEval [Zhou et al., 2023]).

## 6.2 Main Results

Table 1 presents the main results. T2M remasking yields the largest gain on **CMATH** ($+5.92$), a Chinese elementary math benchmark where final answers are exact numbers and even a single

Table 1: Main results across 12 benchmarks. All methods use LLaDA2.1-mini with identical inference parameters. T2M uses LowProb ($\tau$=0.3, $C_{\max}$=1, $\rho_{\max}$=0.25). TriviaQA and DROP report F1; others report accuracy or pass@1 (%). Best per-benchmark in **bold**.

| Category | Benchmark | Original (T2T) | T2M |
|---|---|---|---|
| Knowledge | GPQA-Diamond | **21.72** | **21.72** |
| | TriviaQA (F1) | 30.54 | **30.96** |
| | MMLU-Pro | 58.78 | **58.84** |
| Reasoning | HellaSwag | 78.57 | **78.67** |
| | PIQA | **82.37** | 81.94 |
| | DROP (F1) | 38.53 | **38.90** |
| | BBH | **75.50** | 75.38 |
| Math | CMATH | 82.33 | **88.25** |
| | AIME 2025 | **33.33** | **33.33** |
| | GSM Plus | **67.54** | 67.21 |
| Coding | HumanEval+ | **64.00** | 61.00 |
| Instruction | IFEval (Strict) | 73.01 | **74.12** |

corrupted digit causes failure. Notable improvement is also observed on **instruction following** (IFEval $+1.1$), where precise format compliance benefits from cleaner context. Modest gains appear on **extractive QA** (TriviaQA $+0.42$ F1, DROP $+0.37$ F1), where answers are short spans requiring precise token-level reproduction. On **knowledge** (GPQA $\pm 0.00$, MMLU-Pro $+0.06$), other **mathematics** benchmarks (AIME 2025 $\pm 0.00$ with only 30 problems, GSM Plus $-0.33$), and **commonsense/structured reasoning** (HellaSwag $+0.10$, PIQA $-0.43$, BBH $-0.12$), performance is essentially unchanged. On **coding** (HumanEval+ $-3.0$), performance decreases, likely because remasking disrupts the syntactic token dependencies that code generation relies on—correct code tokens may appear low-confidence in isolation yet be essential for structural validity. We analyze these task-dependent patterns in Section 6.4.

### 6.3 Ablation Studies

**Hyperparameter ablation: cost–accuracy trade-off.** We conduct a full cross-ablation of all hyperparameters on CMATH. We randomly sample 100 problems from the CMATH test set (1,098 problems, fixed seed=42) and run single-sample greedy inference (batch size 1, temperature 0) on a single A100-80GB GPU per configuration. All shared inference parameters follow the Q Mode defaults from our main experiments: M2T confidence threshold $\tau_{\text{m2t}}$=0.7, T2T editing threshold $\tau_{\text{t2t}}$=0.5, block length $B$=32, generation length 16,384.

The M2T fill threshold $\tau_{\text{m2t}}$=0.7 is shared by all configurations (including the baseline), controlling when mask positions are filled. On top of this, we sweep three remask-specific hyperparameters:

1. The *remask threshold* $\tau$ controls how aggressively each strategy triggers remasking. The direction of $\tau$ varies by strategy (Table 2).

| Strategy | Higher $\tau \rightarrow$ | Sweep range |
|---|---|---|
| LowProb | more remasking | $\{0.1, 0.3, 0.5, 0.7, 0.9\}$ |
| T2T-Remask | less remasking | $\{0.5, 0.7, 0.9\}$ |
| LogitDiff | less remasking | $\{0.1, 0.2, 0.3, 0.5\}$ |

Table 2: Effect of increasing $\tau$ on remask volume per strategy.

2. The *per-position budget* $C_{\max} \in \{1, 3, 5\}$ limits how many times each position can be remasked—larger values allow more repeated corrections.
3. The *ratio cap* $\rho_{\max} \in \{0.25, 0.50, 1.0\}$ limits the fraction of editable positions remasked per step—1.0 means no cap.

The baseline uses the original T2T editing with $\tau_{\text{t2t}}$=0.5 (no remasking), with all inference parameters aligned with the official LLaDA2.1-mini Q Mode defaults [Bie et al., 2026]. This yields $1 + (5{+}3{+}4) \times 3 \times 3 = 109$ configurations, all evaluated on the same 100 samples.

Figure 3 plots each configuration as a point. The $x$-axis measures average token modifications per sample—T2T edits for the baseline ($\star$, $\tau_{\text{t2t}}$=0.5, 12.4 edits/sample), remask count for T2M strategies. The $y$-axis measures accuracy.

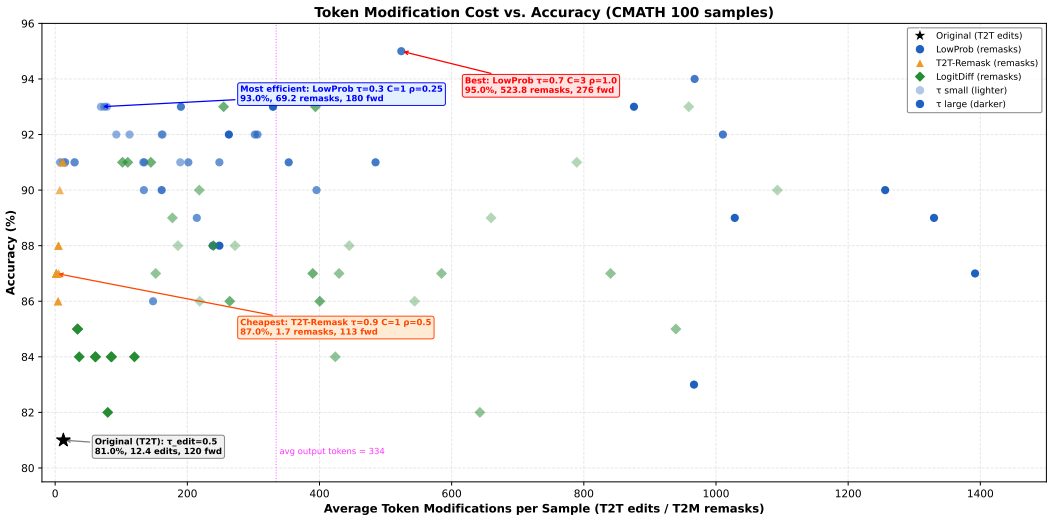


Figure 3: **Token modification cost vs. accuracy.** Each point is one (strategy, $\tau$, $C_{\max}$, $\rho_{\max}$) configuration on CMATH (100 samples). The $x$-axis measures average token modifications per sample: T2T edits for the baseline, remasks for T2M strategies. Color shade encodes $\tau$ (lighter = smaller). The vertical dotted line marks the average output length ($\sim$334 tokens). All T2M strategies surpass the T2T baseline (81%, $\star$). **Most efficient**: LOWPROB $\tau$=0.3, $C$=1, $\rho$=0.25 (93%, only 69.2 remasks). **Cheapest**: T2T-REMASK $\tau$=0.9, $C$=1, $\rho$=0.5 (87%, 1.7 remasks).

Several trends emerge:

- **All strategies improve over T2T.** Every T2M configuration exceeds the T2T baseline (81%, 12.4 edits), confirming that on CMATH, remasking is preferable to token replacement across all tested hyperparameter configurations.
- **LOWPROB dominates.** Across all cost levels, LOWPROB achieves the highest accuracy. The most cost-efficient configuration ($\tau$=0.3, $C$=1, $\rho$=0.25) reaches 93% with only 69.2 remasks.
- **T2T-REMASK is cheapest.** With $\tau$=0.9, $C$=1, $\rho$=0.5, it triggers only 1.7 remasks per sample yet improves accuracy to 87% (+6%).
- **Over-remasking hurts.** Large $\tau$ (e.g., 0.9 for LOWPROB) combined with large $C_{\max}$ and $\rho_{\max}$ produces >1000 remasks per sample, degrading accuracy below 88%. The safety mechanisms ($C_{\max}$, $\rho_{\max}$) are essential for preventing this.

Overall, the most cost-efficient configuration (LOWPROB $\tau$=0.3, $C$=1, $\rho$=0.25) adds only 69.2 remasks per sample—modest relative to the average output length of $\sim$334 tokens—while achieving a 12-point accuracy gain over T2T.

### 6.4 Analysis

**Last-mile token corruption: the dominant error mode.** We conduct a systematic error analysis on CMATH (1,098 elementary math problems), where T2M achieves the largest gain (+5.9%). Among the 194 errors made by T2T, **155 (79.9%)** are *last-mile token corruption*: the model's chain-of-thought reasoning arrives at the correct answer, yet the final extracted answer is corrupted during denoising. Only 39 errors (20.1%) are genuine reasoning failures. Corruption takes several characteristic forms: **extra digits appended** (18 $\rightarrow$ 188, 44 cases), **digits dropped** (32 $\rightarrow$ 2, 26 cases), **answer marker repetition** (25 cases), and **single-digit substitution** (14 cases).

Table 3: Error breakdown on CMATH: T2T vs. T2M. "Corruption" = correct answer in reasoning chain but wrong final extraction.

| | **Total Errors** | **Corruption** | **Genuine Reasoning** | **Accuracy** |
|---|---|---|---|---|
| T2T editing | 194 | 155 (79.9%) | 39 (20.1%) | 82.3% |
| T2M remasking | 129 | 91 (70.5%) | 38 (29.5%) | 88.3% |
| $\Delta$ | $-65$ | $-64$ ($\downarrow$41.3%) | $-1$ | $+5.9\%$ |

T2M remasking repairs **92 of 155** corruption cases (59.4%), reducing total corruption errors from 155 to 91 ($\downarrow$41.3%), while genuine reasoning errors remain virtually unchanged (39$\rightarrow$38). Table 3 provides the full breakdown; representative examples are shown in Figure 4 (Appendix C).

Among the 104 samples that T2M flipped from incorrect to correct, 92 (88.5%) are corruption fixes and only 12 (11.5%) reflect improved reasoning. Conversely, T2M introduces 39 new errors: 30 are new corruption cases, indicating that remasking occasionally destabilizes the denoising process at the answer boundary, and 9 are new reasoning errors where remasking alters the generation path—though both failure modes occur far less often than improvements.

The step-by-step token denoising trajectory for a representative example (DROP 160, gold = 857) is provided in Figure 5 (Appendix D), showing per-token probabilities at each denoising iteration for both T2T and T2M.

## 7 Conclusion

We have identified fundamental limitations of Token-to-Token (T2T) editing in LLaDA2.1 and proposed Token-to-Mask (T2M) remasking as a principled, training-free replacement. By resetting suspected erroneous tokens to the mask state instead of replacing them with potentially incorrect alternatives, T2M purifies the generation context, converts systematic inference errors back to the model's native `[M]` noise type, and enables joint multi-position re-optimization through delayed commitment.

Our theoretical analysis provides a unified understanding of why mask tokens are strictly preferable to erroneous tokens as context, drawing parallels between the discrete and continuous diffusion settings. Three complementary error detection strategies—LowProb, T2T-Remask, and LogitDiff—offer flexibility in detecting different types of token-level errors. Experiments across 12 benchmarks show general improvements on tasks requiring precise token-level output, with the largest gain on mathematics; our error analysis reveals that last-mile token corruption accounts for 79.9% of errors on CMATH, with T2M repairing 59.4% of such corruptions.

**Limitations.** Our approach is specific to the T2T editing mechanism in LLaDA2.1 and may not directly apply to dLLMs without such a mechanism. The theoretical analysis relies on the assumption that mask context is always preferable to erroneous context, which may not hold when error rates are very low. The safety mechanisms introduce hyperparameters ($C_{\max}$, $\rho_{\max}$) that may require task-specific tuning.

**Future work.** A central finding of this work is the *train–inference noise mismatch*: the random token perturbations used in T2T training differ fundamentally from the systematic, model-generated errors encountered at inference. While T2M mitigates this by resetting errors to the model's native `[M]` noise type, a more direct remedy would be to close this gap at the training level. Specifically, *noise-aware continued training* could replace the random perturbations in the T2T training stream with realistic, model-generated errors—collected by running the model's own inference pipeline and recording the actual erroneous tokens that arise. Training on such "real noise" would expose the model to the systematic error patterns it will face at inference, potentially improving both M2T predictions under polluted context and T2T correction accuracy. This is complementary to our inference-time approach: T2M improves the correction *mechanism*, while noise-aware training improves the model's *robustness* to realistic errors.

Beyond this, combining T2M remasking with sampling-level remasking (e.g., ReMDM) could yield further improvements. Learning the remask threshold adaptively (e.g., via the PRISM framework) rather than using fixed heuristics is a promising direction. Extending the analysis to other dLLM architectures beyond LLaDA2.1 is also of interest.

## Acknowledgments and Disclosure of Funding

Anonymous for review.

## References

Jacob Austin, Daniel Johnson, Jonathan Ho, Daniel Tarlow, and Rianne van den Berg. Structured denoising diffusion models in discrete state-spaces. In *Advances in Neural Information Processing Systems*, 2021.

Tiwei Bie, Maosong Cao, Xiang Cao, Bingsen Chen, et al. LLaDA2.1: Speeding up text diffusion via token editing. *arXiv preprint arXiv:2602.08676*, 2026.

Yonatan Bisk et al. PIQA: Reasoning about physical intuition in natural language. *arXiv preprint arXiv:1911.11641*, 2020.

Mark Chen et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Dheeru Dua et al. DROP: A reading comprehension benchmark requiring discrete reasoning over paragraphs. *arXiv preprint arXiv:1903.00161*, 2019.

Haoyu He, Katrin Renz, Yong Cao, and Andreas Geiger. MDPO: Overcoming the training-inference divide of masked diffusion language models. *arXiv preprint arXiv:2508.13148*, 2025.

Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems*, 2020.

Zemin Huang, Yuhang Wang, Zhiyang Chen, and Guo-Jun Qi. Don't settle too early: Self-reflective remasking for diffusion language models. *arXiv preprint arXiv:2509.23653*, 2025.

Mandar Joshi, Eunsol Choi, Daniel S Weld, and Luke Zettlemoyer. TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension. *arXiv preprint arXiv:1705.03551*, 2017.

Wonjun Kang, Kevin Galim, Seunghyuk Oh, Minjae Lee, Yuchen Zeng, Shuibai Zhang, Coleman Hooper, Yuezhou Hu, Hyung Il Koo, Nam Ik Cho, and Kangwook Lee. ParallelBench: Understanding the trade-offs of parallel decoding in diffusion LLMs. *arXiv preprint arXiv:2510.04767*, 2025.

Jaeyeon Kim, Seunggeun Kim, Taekyun Lee, David Z Pan, Hyeji Kim, Sham Kakade, and Sitan Chen. Fine-tuning masked diffusion for provable self-correction. *arXiv preprint arXiv:2510.01384*, 2025.

Qintong Li, Leyang Cui, Xueliang Zhao, Lingpeng Kong, and Wei Bi. GSM-Plus: A comprehensive benchmark for evaluating the robustness of LLMs as mathematical problem solvers. *arXiv preprint arXiv:2402.19255*, 2024.

Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation. In *Advances in Neural Information Processing Systems*, 2023.

Aaron Lou, Chenlin Meng, and Stefano Ermon. Discrete diffusion modeling by estimating the ratios of the data distribution. In *International Conference on Machine Learning*, 2024.

Mathematical Association of America. American invitational mathematics examination 2025, 2025.

Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Zhou, Ji-Rong Wen, and Chongxuan Li. LLaDA: Large language diffusion with masking. In *International Conference on Machine Learning*, 2025.

Jingyang Ou, Shen Nie, Kaiwen Xue, Fengqi Zhu, Jiacheng Wen, Chongxuan Li, and Zhizhou Zhang. Your absorbing discrete diffusion secretly models the conditional distributions of clean data. In *International Conference on Learning Representations*, 2025.

David Rein et al. GPQA: A graduate-level Google-proof Q&A benchmark. *arXiv preprint arXiv:2311.12022*, 2024.

Subham Sekhar Sahoo, Marianne Arriola, Yair Schiff, Aaron Gokaslan, Edgar Marroquin, and Volodymyr Kuleshov. Simple and effective masked diffusion language models. In *Advances in Neural Information Processing Systems*, 2024.

Yair Schiff, Omer Belhasin, Roy Uziel, Guanghan Wang, Marianne Arriola, Gilad Turok, Michael Elad, and Volodymyr Kuleshov. Learn from your mistakes: Self-correcting masked diffusion models. *arXiv preprint arXiv:2602.11590*, 2026.

Yang Song, Jasper Sohl-Dickstein, Diederik P Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. In *International Conference on Learning Representations*, 2021.

Mirac Suzgun, Nathan Scales, Nathanael Schärli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc V Le, Ed H Chi, Denny Zhou, and Jason Wei. Challenging BIG-Bench tasks and whether chain-of-thought can solve them. *Findings of ACL*, 2023.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.

Guanghan Wang, Yair Schiff, Subham Sekhar Sahoo, and Volodymyr Kuleshov. Remasking discrete diffusion models with inference-time scaling. In *Advances in Neural Information Processing Systems*, 2025.

Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyan Jiang, et al. MMLU-Pro: A more robust and challenging multi-task language understanding benchmark. *arXiv preprint arXiv:2406.01574*, 2024.

Tianwen Wei, Jian Luan, Wei Liu, Shuang Dong, and Bin Wang. CMATH: Can your language model pass chinese elementary school math test? *arXiv preprint arXiv:2306.16636*, 2023.

Jiacheng Ye, Zhihui Xie, Lin Zheng, Jiahui Gao, Zirui Wu, Xin Jiang, Zhenguo Li, and Lingpeng Kong. Dream 7B: Diffusion large language models. *arXiv preprint arXiv:2508.15487*, 2025.

Rowan Zellers et al. HellaSwag: Can a machine really finish your sentence? *arXiv preprint arXiv:1905.07830*, 2019.

Kevin Zhai, Sabbir Mollah, Zhenyi Wang, and Mubarak Shah. CORE: Context-robust remasking for diffusion language models. *arXiv preprint arXiv:2602.04096*, 2026.

Jeffrey Zhou et al. Instruction-following evaluation for large language models. *arXiv preprint arXiv:2311.07911*, 2023.

## A Benchmark Details

Table 4 lists all benchmarks used in our evaluation with dataset details.

Table 4: Benchmark details.

| Category | Benchmark | Split | Samples | Metric |
|---|---|---|---|---|
| Knowledge | GPQA-Diamond | train | 198 | Accuracy |
| | TriviaQA | val | 17,944 | EM / F1 |
| | MMLU-Pro | test | 12,032 | Accuracy |
| Reasoning | HellaSwag | val | 10,042 | Accuracy |
| | PIQA | test | 1,838 | Accuracy |
| | DROP | val | 9,535 | EM / F1 |
| | BBH | test | 6,511 | Accuracy |
| Math | CMATH | test | 1,098 | EM (numeric) |
| | AIME 2025 | – | 30 | EM (integer) |
| | GSM Plus | test | 2,400 | EM (numeric) |
| Coding | HumanEval+ | – | 164 | pass@1 |
| Instruction | IFEval | test | 541 | Prompt Strict Acc |

## B Hyperparameter Details

All remask-specific hyperparameters and their defaults are listed in the main text. Shared inference parameters follow LLaDA2.1 Q Mode: `threshold`$= 0.7$, `editing_threshold`$= 0.5$, `block_length`$= 32$, `temperature`$= 0.0$. The denoising inner loop runs until convergence (all masks filled and no edits/remasks triggered), not for a fixed number of steps.

## C Representative Case Studies on CMATH

Figure 4 shows a representative CMATH case from the corruption analysis in Section 6. The model performs correct arithmetic in its chain-of-thought, but T2T editing fails to prevent the final answer from being corrupted. T2M remasking, by resetting low-confidence tokens to `[M]` and re-predicting them, produces the correct final answer.

**Leading digits dropped + answer marker repeated** Gold: **31**

*Q: Xiaofang is 8 years old and her mother is 39. How many years younger is Xiaofang than her mother?*

**T2T**: Reasoning: $39 - 8 = 31$ ✓ → "The answer is the answer is **1**" ✗

**T2M**: Reasoning: $39 - 8 = 31$ ✓ → "The answer is **31**" ✓

Figure 4: Representative CMATH case illustrating last-mile token corruption. Both methods produce correct chain-of-thought reasoning, but T2T editing generates a corrupted final answer—with a dropped leading digit and repeated answer marker—while T2M remasking outputs the correct result. The answer marker phrase is translated from the Chinese original for readability.

## D Token Denoising Trajectory

Figure 5 shows the full token-level denoising trajectory for the DROP 160 example (gold = 857) discussed in Section 6.4. Each cell displays the token and its model probability at that denoising step. The figure corresponds to Figure 1(b) and provides the step-by-step detail of how T2T commits the error "8"→"6" at $t$=1 while T2M remasks and recovers "8" at $t$=2.

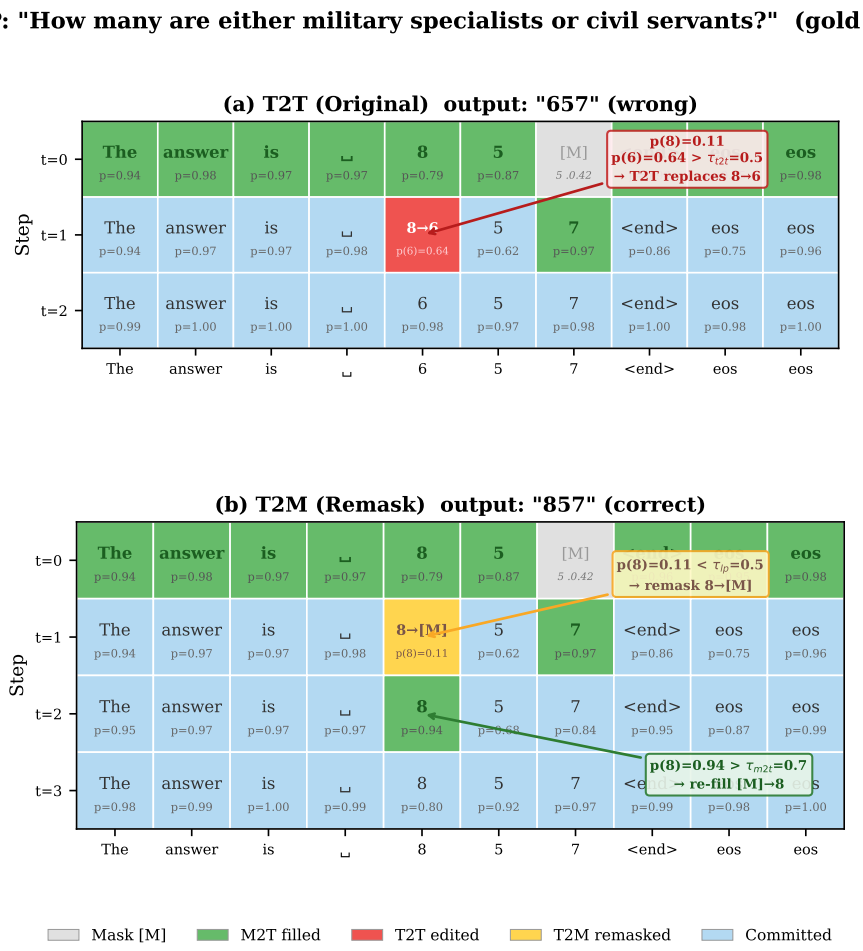


Figure 5: **Token denoising trajectory** for DROP example 160 (gold = 857). Each cell shows the token and its probability. **(a)** T2T: "8" ($p$=0.11) is replaced by "6" ($p$=0.64) at $t$=1, producing 657. **(b)** T2M: "8" is remasked at $t$=1, re-predicted as "8" ($p$=0.94) at $t$=2 under converged context ("5", "7" committed), producing 857.